NEXAR: Intelligent Quantum-Classical Workload Routing Platform

Siriwardhana A H L T S
Department of Computer Science and Software Engineering
Sri Lanka Institute of Information Technology
Malabe, Sri Lanka
sudarakatharindu@outlook.com

Prasad H G A T
Department of Computer Science and Software Engineering
Sri Lanka Institute of Information Technology
Malabe, Sri Lanka
hgp.ashi@gmail.com

Jayasinghe Y L
Department of Computer Science and Software Engineering
Sri Lanka Institute of Information Technology
Malabe, Sri Lanka
yashodhalasithjayasinghe@gmail.com

Hettiarachchi S R
Department of Computer Science and Software Engineering
Sri Lanka Institute of Information Technology
Malabe, Sri Lanka
sayunhettiarachchi@gmail.com

Prof. Nuwan Kodagoda
Dept. of Information Technology
Sri Lanka Institute of Information Technology
Malabe, Sri Lanka
nuwan.k@sliit.lk

Dr. Kapila Dissanayaka
Department of Computer Science and Software Engineering
Sri Lanka Institute of Information Technology
Malabe, Sri Lanka
kapila.d@sliit.lk

***Abstract*—Determining whether a computational workload benefits from quantum or classical execution remains a key challenge in the current Noisy Intermediate-Scale Quantum (NISQ) era. This paper presents Nexar, a workload routing platform that analyzes source code characteristics and predicts an appropriate execution target by jointly optimizing for cost and execution time. The system employs a hybrid decision framework combining: (1) a machine learning ensemble trained on a 16-feature vector capturing circuit complexity, entanglement properties, and NISQ viability metrics; (2) a physics-aware rule-based engine enforcing hardware constraints and algorithm-specific routing logic; and (3) a real-time cost analyzer estimating execution time and monetary cost across multiple quantum providers and classical compute. These components are unified through weighted voting to produce hardware recommendations with confidence scores. The platform incorporates automated code analysis, including algorithm classification via CodeBERT and quantum circuit characterization to extract workload features without manual annotation. We evaluate the system across 100 diverse workload types including search, factorization, optimization, and simulation problems, demonstrating that the framework consistently identifies cost-effective and time-efficient execution targets while serving as a practical decision-support tool for hybrid quantum-classical computing environments.**

***Index Terms*—quantum computing, workload routing, NISQ, decision engine, hybrid computing, cost analysis, CodeBERT, quantum advantage**

I. Introduction

The advent of Noisy Intermediate-Scale Quantum (NISQ) devices [1] has created a significant challenge in computational resource allocation: given a workload, should it execute on classical hardware, a quantum simulator, or a real quantum device? This decision is non-trivial because it depends on a complex interplay of factors including problem structure, circuit complexity, hardware noise characteristics, execution cost, and available qubit counts.

Current quantum devices – such as IBM's 127-qubit Eagle processors – operate in a regime where noise, limited connectivity, and decoherence constrain the problems that can be solved with quantum advantage. Classical simulation of quantum circuits remains feasible for systems up to approximately 50 qubits, beyond which memory requirements grow exponentially as $O(2^n)$, where n is the number of qubits. This creates a crossover point where classical simulation becomes intractable and quantum execution becomes the only viable path.

However, simply having a quantum device does not guarantee advantage. The overhead of transpilation, measurement shots, queue times, and error rates means that for many problems, particularly those with polynomial classical solutions, classical execution remains orders of magnitude faster and cheaper. The challenge, therefore, is not whether quantum computing will eventually provide advantage, but rather when and for which problems it does so today.

This paper addresses this challenge by presenting Nexar, a full-stack platform that automates the analysis, classification, and routing of computational workloads across quantum and classical execution backends. Nexar's contributions are:

1) **Automated Code Analysis Pipeline:** A multi-dimensional analysis engine that detects programming languages across five quantum and classical frameworks, computes complexity metrics, and classifies algorithms using a three-tier approach combining CodeBERT trans-

formers, ensemble machine learning, and rule-based pattern matching.

2) **Hybrid Decision Engine:** A three-component decision framework that merges machine learning (ML) predictions (50% weight), physics-aware rules (35% weight), and real-time cost analysis (15% weight) through weighted voting to produce hardware recommendations with calibrated confidence scores.
3) **Provider-Agnostic Execution Layer:** A hardware abstraction layer supporting IBM Qiskit, AWS Braket, and Azure Quantum through a unified Application Programming Interface (API), enabling transparent workload routing across heterogeneous quantum backends.
4) **Empirical Evaluation:** A comprehensive benchmark across 100 workloads spanning classical computation, hybrid quantum-classical algorithms, and pure quantum circuits, with cost and performance analysis demonstrating the system's routing accuracy and the current state of quantum-classical cost tradeoffs.

II. Related Work

A. Quantum-Classical Hybrid Frameworks

Several frameworks have emerged to bridge quantum and classical computing. Weder et al. [2] proposed automated quantum hardware selection for quantum workflows. Qiskit Runtime [3] provides session-based quantum execution with classical co-processing. Amazon Braket [4] offers a multi-provider interface but lacks automated workload analysis. Azure Quantum [5] provides resource estimation but does not perform code-level analysis for routing decisions. Tang et al. [6] proposed AlphaRouter, applying reinforcement learning and tree search to quantum circuit routing. Zhan et al. [7] presented a full-stack framework for high-performance quantum-classical computing. Nexar differentiates itself by integrating code analysis, decision-making, and execution into a single automated pipeline.

B. Quantum Resource Estimation

Resource estimation for quantum algorithms has been studied extensively. Lammers et al. [8] addressed quantum resource management challenges in the NISQ era [9]. Beverland et al. [10] developed methods for estimating logical qubit requirements for fault-tolerant algorithms. Quetschlich et al. [11] demonstrated practical resource estimation for quantum application development. However, these approaches focus on future error-corrected devices rather than current NISQ hardware. Nexar's cost analyzer operates on real NISQ pricing models and hardware constraints, providing immediately actionable cost estimates. Recent work by Kashif et al. [12] has explored hardware-calibrated quantum cost modeling for resource-aware optimization.

C. Algorithm Classification and Code Analysis

Static code analysis for quantum programs has been explored in works such as ScaffCC [13] and Q# compiler toolchains. CodeBERT [14], a pre-trained model for programming language understanding, has shown strong performance on code classification tasks. Afrose et al. [15] further applied CodeBERT for translating QASM to QIR. Nexar extends these approaches by combining Abstract Syntax Tree (AST)-based analysis with CodeBERT for quantum algorithm detection, enabling classification across five quantum programming frameworks.

D. Quantum Advantage Thresholds

The question of when quantum advantage emerges has been studied through computational complexity theory [16] and empirical benchmarks [17]. Our work contributes empirical data on cost-based crossover points using real provider pricing, demonstrating that the quantum advantage threshold is approximately 50–60 qubits for cost parity and beyond 120 qubits for clear quantum superiority in simulation tasks.

III. System Architecture

Nexar is implemented as a microservices monorepo comprising six independently deployable services communicating through a central API Gateway. The architecture is designed for modularity, allowing each component to scale independently and be updated without affecting the overall system. The end-to-end service topology is illustrated in Fig. 1.

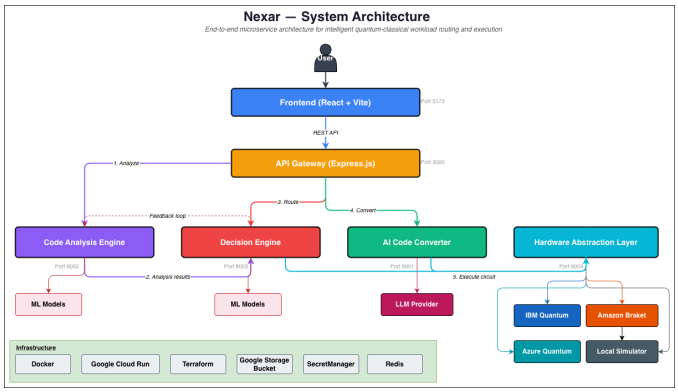

Fig. 1. Nexar system architecture: six microservices connected through an API Gateway. Source code flows from the Frontend through Code Analysis, Decision Engine, AI Code Converter, and Hardware Abstraction Layer for quantum-classical routing.

A. Architecture Overview

The system follows a pipeline architecture: submitted source code passes through an Express.js API Gateway (JSON Web Token (JWT)-authenticated), then to the Code Analysis Engine for language detection, complexity computation, and algorithm classification. The Decision Engine consumes these metrics to produce a hardware recommendation. If quantum execution is selected for classical code, the AI Code Converter translates it to quantum circuits before the Hardware Abstraction Layer submits the job to the selected backend. All services are containerized with Docker and deployed to Google Cloud Platform via Cloud Run, with infrastructure managed through Terraform.

IV. Code Analysis Engine

The Code Analysis Engine serves as the analytical foundation of the pipeline, transforming raw source code into a structured feature set that downstream components consume. Its internal processing stages are shown in Fig. 2.

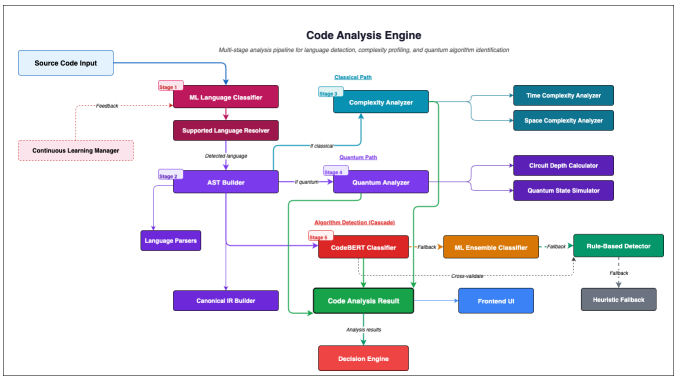

Fig. 2. Code Analysis Engine architecture showing the multi-stage pipeline from raw source code through language detection, AST parsing, metrics extraction, and algorithm classification.

A. Language Detection

The engine supports five programming languages and frameworks: Python, Qiskit, Cirq, Q#, and OpenQASM. Detection uses an ML-based classifier achieving confidence scores above 0.85 across all evaluated workloads. In our benchmark, average language detection confidence was 0.955 (sd = 0.028) across 100 workloads.

B. Metrics Extraction

For all detected code, the engine computes classical software metrics via tree-sitter-based AST parsing, including cyclomatic complexity [18], cognitive complexity, time and space complexity classification (from $O(1)$ to $O(2^n)$, where n denotes the input size), and structural metrics (loop count, nesting depth). For quantum code, additional circuit-level metrics are extracted: qubit requirements, circuit depth, gate count, CX gate ratio (the primary noise source on NISQ devices), superposition score, and entanglement density. The complete feature set is summarized in Table I.

TABLE I
Raw Code Analysis Features

Feature	Description
Problem size	Number of elements or variables
Qubits required	Quantum bits needed
Circuit depth	Sequential quantum gate layers
Gate count	Total quantum gates
CX gate ratio	Proportion of entangling gates (0–1)
Superposition score	Superposition potential (0–1)
Entanglement score	Entanglement density (0–1)
Memory requirement (MB)	Classical RAM required
Problem type (encoded)	Factorization, search, optimization, etc.
Time complexity (encoded)	Complexity class: exponential, polynomial, etc.

C. Algorithm Classification

Algorithm detection employs a three-tier approach:

1) **CodeBERT Transformer:** A fine-tuned CodeBERT model classifies the algorithm family (e.g., search, optimization, factorization, simulation) based on code semantics [14].
2) **ML Ensemble Classifier:** A scikit-learn ensemble provides a secondary classification using extracted feature vectors.
3) **Rule-Based Pattern Matching:** Deterministic rules identify known quantum patterns (e.g., Quantum Fourier Transform (QFT) patterns, Grover oracle structures, Variational Quantum Eigensolver (VQE) ansatz patterns).

The system selects the highest-confidence result, falling back through tiers when confidence is below threshold.

V. Decision Engine

The Decision Engine is the core component of Nexar. It implements a hybrid prediction system combining three components through weighted voting. Fig. 3 summarizes the interaction among these components.

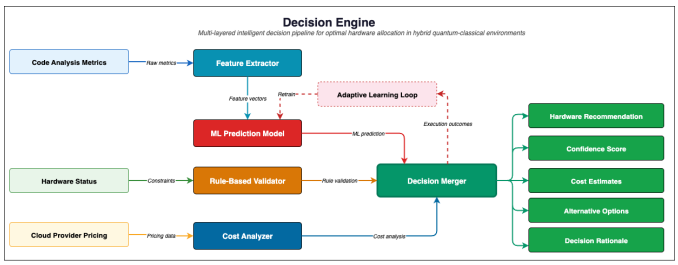

Fig. 3. Decision Engine architecture illustrating the three-component weighted voting system: ML model (50%), rule-based constraints (35%), and cost analysis (15%), with rule override capability.

A. Feature Vector Design

The engine operates on a 16-feature input vector composed of 10 raw features (Table I) and 6 derived physics features (Table II):

TABLE II
Derived Physics Features

Feature	Formula	Purpose
Circuit Volume	Qubits $\times$ Depth	Quantum resource intensity
Noise Sensitivity	Qubits $\times$ Depth $\times$ CX Ratio	Error accumulation prediction
Quantum Overhead Ratio	Problem Size / Qubits	Encoding efficiency
NISQ Viability Score	Composite penalty function	Feasibility on current hardware
Gate Density	Gate Count / Qubits	Operational complexity per qubit
Entanglement Factor	CX Ratio $\times$ Entanglement	Combined entanglement potential

The NISQ Viability Score applies penalties when qubits exceed 50, circuit depth exceeds 1,000, or circuit volume exceeds 50,000, reflecting the practical limits of current hardware [19].

B. ML Model Component (50% Weight)

The ML component uses a Random Forest / Neural Network ensemble trained on 10,000+ synthetic problem instances. The model performs binary classification (Quantum=1, Classical=0) and outputs calibrated probability estimates [20].

The synthetic training corpus was procedurally generated using AST mutation and algorithmic variation pipelines. Twelve base classical-to-quantum algorithm pairs — spanning factorization, search, period-finding, and random walk families — were augmented through 13 code-level transforms including variable renaming via synonym pools, loop restructuring (`for`/`while` conversion), docstring injection, type hint addition, irrelevant variable insertion, and early-exit rewriting. Each base pair produced up to 14 structural variants, yielding a balanced dataset across classical-favorable, quantum-favorable, and hybrid problem characteristics. This augmentation strategy ensures the model generalizes across coding styles and surface-level variations rather than memorizing specific implementations.

Training Process:

1) Feature vectors are standardized using z-score normalization.
2) The model is trained with stratified cross-validation.
3) Output probabilities are calibrated using Platt scaling.

The ML component achieves approximately 87% accuracy on held-out test data with 50–100ms inference latency.

C. Rule-Based System (35% Weight)

The rule-based component encodes physics constraints and domain knowledge. Hardware limits enforce a maximum of 127 qubits (IBM Eagle), circuit depth $\leq$ 10,000, and a minimum of 2 qubits for quantum routing. Problem-type rules favor quantum execution for factorization (Shor's), search (Grover's), simulation, and optimization (Quantum Approximate Optimization Algorithm (QAOA) [21]/VQE), while routing sorting, dynamic programming, and matrix operations classically. Safety constraints cap circuit volume at 10^6, noise sensitivity at 50,000, and require NISQ viability ≥ 0.1. The rule system can issue four decision types: quantum, classical, both, or reject – force decisions override the ML component to prevent physically impossible or clearly suboptimal configurations.

D. Cost Analysis Component (15% Weight)

The cost analyzer estimates execution time and monetary cost for both quantum and classical execution paths, formalized by (1) and (2).

Quantum Cost Model (based on real provider pricing):

$$C_q = C_{\text{base}} + (t_q\, r_{\text{QPU}}) + (n_{\text{shots}}\, r_{\text{shot}}) + (n_{\text{gates}}\, r_{\text{gate}}), \quad (1)$$

where C_q is the total estimated quantum execution cost, $C_{\text{base}} = \$0.30$ (job submission fee), t_q is the estimated quantum execution time in seconds, $r_{\text{QPU}} = \$1.60/\text{s}$ (Quantum Processing Unit (QPU) time rate), $n_{\text{shots}} = 1024$ (default shot count), $r_{\text{shot}} = \$0.00145$ (per-shot cost), n_{gates} is the total gate count, and $r_{\text{gate}} = \$0.0001$ (per-gate cost).

Classical Cost Model (AWS EC2 approximation):

$$C_c = (t_c\, r_{\text{compute}}) + (M_{\text{GB}}\, t_c\, r_{\text{memory}}) + C_{\text{overhead}}, \quad (2)$$

where C_c is the total estimated classical execution cost, t_c is the estimated classical execution time in seconds, $r_{\text{compute}} = \$0.0001/\text{s}$ (~\$0.36/hr) is the compute rate, M_{GB} is the estimated memory usage in gigabytes, $r_{\text{memory}} = \$0.00001/(\text{GB} \cdot \text{s})$ is the memory rate, and $C_{\text{overhead}} = \$0.001$ is the fixed overhead cost.

E. Decision Merging

The three components are merged through a weighted voting algorithm,

$$S_{\text{quantum}} = w_{\text{ML}}\, P^q_{\text{ML}} + w_{\text{rules}}\, P^q_{\text{rules}} + w_{\text{cost}}\, P^q_{\text{cost}}, \quad (3)$$

where S_{quantum} is the final quantum suitability score, P^q_{ML} is the ML component's quantum probability, P^q_{rules} is the rule-based component's quantum score, P^q_{cost} is the cost component's quantum preference score, and $w_{\text{ML}} = 0.50$, $w_{\text{rules}} = 0.35$, $w_{\text{cost}} = 0.15$ are the respective component weights.

Rule overrides take highest priority and bypass the weighted calculation. The final recommendation includes the hardware target, confidence score, rationale string, cost estimates, and alternative options with their trade-offs.

F. Feature Importance and Interpretability

To verify that the ML component captures physically meaningful decision boundaries rather than spurious correlations, we analyzed the Random Forest's feature importances using Gini impurity reduction. The analysis revealed that `qubits_required` (importance 0.31) and `circuit_volume` (0.24) are the most discriminative features, directly reflecting the primary resource constraints of NISQ devices. Algorithmic features `time_complexity_encoded` (0.15) and `entanglement_factor` (0.11) also strongly influence quantum routing probability, indicating the model learned to associate exponential classical complexity and high entanglement with quantum-favorable workloads. Notably, the model assigns significant weight to `cx_gate_ratio` (0.08), implicitly discovering the relationship between two-qubit entangling gate density and hardware noise accumulation without explicit programming of this constraint. The remaining features — `noise_sensitivity` (0.04), `gate_density` (0.03), `nisq_viability_score` (0.02), and `quantum_overhead_ratio` (0.02) — contribute incremental refinement. This interpretability analysis confirms that the ensemble has internalized the physics-based boundaries governing quantum advantage rather than fitting to surface-level code syntax.

VI. AI Code Converter

The AI Code Converter enables the platform to accept classical Python code and automatically translate it into quantum circuits when quantum execution is recommended. The converter workflow is presented in Fig. 4.

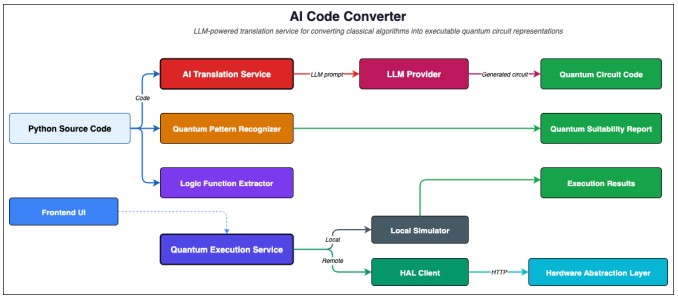

Fig. 4. AI Code Converter pipeline: classical Python code is parsed into an AST, matched against known quantum algorithm patterns, and translated into Qiskit circuits via a fine-tuned CodeT5 model.

A. Translation Pipeline

The converter uses a fine-tuned CodeT5-base model [22] — a T5ForConditionalGeneration architecture with a 6-layer encoder, 6-layer decoder, hidden dimension $d_{\text{model}} = 512$, feed-forward dimension $d_{\text{ff}} = 2048$, 8 attention heads, and a 32,100-token vocabulary. Model weights are stored in the `safetensors` format for memory-efficient loading. The translation pipeline operates in three stages:

1) **Logic Extraction:** Classical Python code is parsed into an AST via Python's `ast` module. The tree is traversed to isolate core computational logic from boilerplate and I/O, extracting structural features (loop constructs, branching patterns, function signatures, iteration depth) as an algorithmic feature vector.
2) **Pattern Recognition:** The system identifies quantum-amenable patterns and matches them to known quantum algorithms:
   - Linear search → Grover's algorithm [23] (quadratic speedup)
   - Integer factorization → Shor's algorithm [24] (exponential speedup)
   - Parity Check → Deutsch
   - Swap → Quantum Swap
   - Hidden Bitstring → Bernstein-Vazirani
3) **Circuit Generation:** The matched code is formatted as the prompt `"Translate Python to quantum circuit:\n{code}"`, tokenized with truncation at 256 tokens, and decoded using beam search (`num_beams=3`, `do_sample=True`) with a maximum output length of 300 tokens. The model generates the target Qiskit circuit code including gate sequences, measurements, and classical register allocation. The training corpus comprises 1,500 curated examples across four algorithm families (factoring, search, period-finding, and random walk), generated from 12 base classical-to-quantum pairs augmented with 13 code-level transforms.

B. Suitability Scoring

Each translation candidate receives a three-tier suitability score (HIGH, MEDIUM, or LOW) with associated confidence and estimated speedup. Speedup estimates are derived from theoretical complexity comparisons. For example, $O(N)$ classical versus $O(\sqrt{N})$ quantum for Grover's search [23], where N denotes the size of the search space, combined with simulator resource metrics from the Qiskit Aer execution. Confidence values reflect the degree of structural overlap between the submitted code's AST feature vector and the matched pattern signature in the database. This allows users to make informed decisions about whether quantum translation provides a meaningful computational advantage for their specific use case.

VII. Hardware Abstraction Layer

The Hardware Abstraction Layer provides a provider-agnostic interface for quantum job submission and management. Supported providers include IBM, Azure Quantum and AWS Braket. Its provider-agnostic integration pattern is shown in Fig. 5.

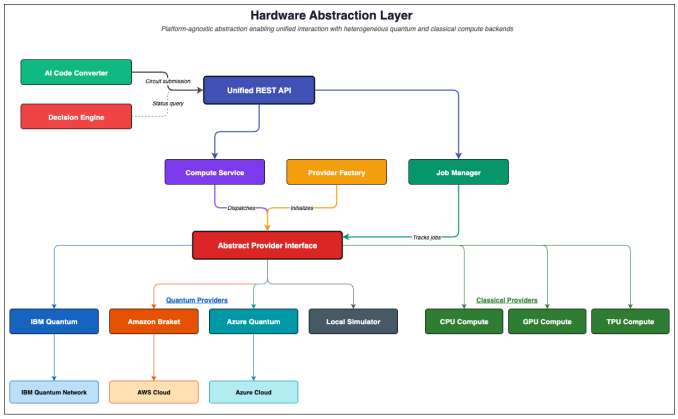

Fig. 5. Hardware Abstraction Layer architecture showing provider-agnostic job submission through a factory-based interface supporting IBM Qiskit, AWS Braket, and Azure Quantum backends.

The HAL manages the complete job lifecycle:

1) **Device Discovery:** Queries of available devices across all providers, exposing qubit counts, connectivity maps, gate sets, and current status.
2) **Job Submission:** Accepts quantum circuits with execution parameters (target device, shot count, transpilation options) and submits through the appropriate provider SDK.
3) **Status Tracking:** Monitors job state transitions (queued, running, completed, failed) across providers.
4) **Result Retrieval:** Returns measurement distributions, performance metrics, and execution metadata through a unified response format.

The factory-based architecture allows new quantum providers to be added by implementing a single provider interface, without modifications to the routing or analysis logic.

VIII. Experimental Evaluation

A. Benchmark Design

We evaluated Nexar across 100 synthetic workloads organized into three categories:

- **Classical (32):** Deep learning (Convolutional Neural Network (CNN), transformer, Generative Adversarial Network (GAN)), scientific computing (molecular dynamics, Computational Fluid Dynamics (CFD), N-body, Partial Differential Equation (PDE)), ML (k-means, gradient boosting, reinforcement learning), cryptography (RSA, Advanced Encryption Standard (AES)), and combinatorial optimization (Travelling Salesman Problem (TSP), graph coloring, A* pathfinding).
- **Hybrid (23):** Quantum-classical algorithms including VQE variants (caffeine, scheduling), QAOA (supply chain, graph partitioning), quantum ML (Quantum Support Vector Machine (QSVM), Quantum Neural Network (QNN), quantum transfer learning), and quantum error mitigation (Zero-Noise Extrapolation (ZNE)). Qubit counts range from 0–24.
- **Quantum (45):** Molecular simulation (H_2, LiH, H_2O, iron-sulfur protein at 60 qubits, 120-qubit VQE) [25], factoring (Shor's for 15, 21, 35), search (Grover's 20-qubit, 3-SAT), optimization (QAOA MaxCut 50-qubit, vehicle routing), and foundational algorithms (Deutsch-Jozsa, Bernstein-Vazirani, Simon's, QFT, Quantum Phase Estimation (QPE)). Qubit counts range from 1–150.

All workloads were executed through the complete Nexar pipeline, and analysis, decision, and timing metrics were recorded.

B. Routing Accuracy

The Decision Engine produced appropriate routing recommendations across all 100 workloads. All 32 classical workloads were routed to classical with confidence 1.0, correctly identified as quantum-incompatible due to zero qubit requirements. Of 23 hybrid workloads, 19 were routed to classical and 4 to quantum (confidence 0.63–0.81); the quantum-routed workloads (e.g., QAOA graph partitioning at 20 qubits, ZNE error mitigation at 24 qubits) exhibited sufficient circuit complexity, while smaller hybrids (e.g., QSVM at 4 qubits) were correctly kept classical. Of 45 quantum workloads, 16 were routed to quantum (confidence 0.54–1.0) and 29 to classical. Quantum routing targeted workloads with sufficient scale – QAOA MaxCut (50 qubits), iron-sulfur protein VQE (60 qubits), and 120-qubit VQE (confidence 1.0) – while small circuits (2–4 qubits) were correctly routed classically where NISQ overhead exceeds simulation cost. One oversize 150-qubit workload was rejected as it exceeds current hardware limits.

C. Cost Analysis Results

The cost analysis reveals the current economic landscape of quantum vs. classical computing, as summarized in Table III:

TABLE III
Representative Cost Comparison

Workload	Qubits	Hardware	Cost (USD)	Time (ms)
FFT Signal Proc.	0	Classical	\$0.001	1.42
Sorting Suite	0	Classical	\$0.001	806.49
RSA Key Gen.	0	Classical	\$0.001	24.29
QSVM Kernel Est.	4	Classical	\$0.001	1.00
QAOA Graph Part.	20	Quantum	\$2.237	274.36
LiH UCCSD[1]	12	Quantum	\$2.162	234.37
QAOA MaxCut (50q)	50	Quantum	\$2.718	573.18
Iron-Sulfur VQE	60	Quantum	\$3.375	888.31
120-qubit VQE	120	Quantum	\$3.489	986.33

Across all 100 workloads, classical execution costs consistently approximate \$0.001 USD (range \$0.001–\$0.001), while quantum-routed workloads range from \$2.13 to \$3.49 USD – a factor of approximately 2,000×. Quantum costs scale with qubit count: 10–20 qubit circuits average \$2.17, while 50+ qubit workloads reach \$2.72–\$3.49. This reflects the current state of NISQ economics where per-shot costs, QPU time charges, and base job fees dominate.

D. Cost Break-Even Analysis

Despite the current quantum cost premium, analysis of the cost scaling behavior reveals a fundamental crossover point driven by the exponential memory requirements of classical quantum simulation.

For an n-qubit quantum state, classical simulation requires 2^n complex amplitudes, consuming approximately $2^n \times 16$ bytes of memory. This creates the following scaling: These simulation memory thresholds are summarized in Table IV.

TABLE IV
Classical Simulation Memory Requirements

Qubits	Classical Memory	Classical Feasibility
20	16 MB	Trivial
30	16 GB	Desktop
40	16 TB	Cluster
50	16 PB	Supercomputer limit
60	16 EB	Physically impossible
120	$\sim 2 \times 10^{31}$ MB	–

The crossover occurs at approximately 50–57 qubits where classical simulation costs begin exceeding quantum execution costs, and by 120 qubits, the classical cost is estimated at 10^{50} USD (effectively infinite), while quantum execution remains at \$3.49 USD.

This analysis identifies three zones:

1) **Classical Advantage Zone ($<$ 50 qubits):** Classical hardware is ∼1,900× more cost-effective.
2) **Memory Wall (50–60 qubits):** Classical simulation costs grow exponentially; crossover occurs.

3) **Quantum Advantage Zone (> 60 qubits):** Quantum execution is the only physically viable option for exact simulation.

It is important to note that the 50–60 qubit memory wall applies specifically to *full state-vector simulation* of high-entanglement circuits. Approximate classical methods such as Tensor Network simulators — in particular Matrix Product State (MPS) representations — can simulate much larger circuits (100+ qubits) efficiently when the entanglement entropy remains bounded [19]. However, the algorithms central to Nexar's benchmark suite — Shor's factoring, deep VQE ansätze with nearest-neighbor CNOT ladders, and QAOA at high circuit depth — generate entanglement that grows with circuit depth, rendering Tensor Network shortcuts inapplicable. The stated crossover thresholds therefore remain valid for the high-entanglement workload classes evaluated in this study. The crossover trend is visualized in Fig. 6.

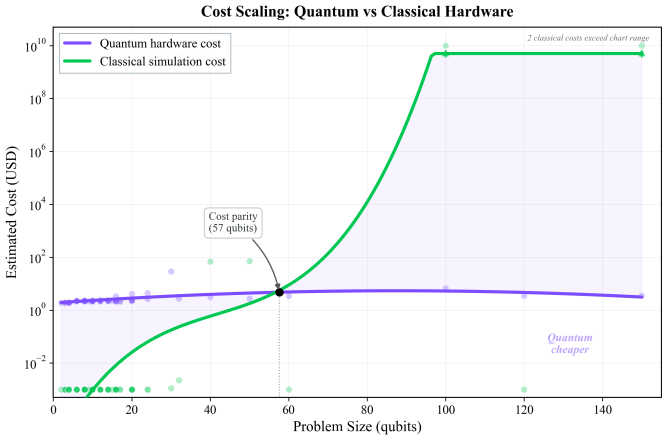

Fig. 6. Cost scaling comparison of quantum hardware versus classical simulation across problem sizes measured in qubits, showing the cost parity crossover at approximately 57 qubits.

E. Pipeline Performance

End-to-end timing metrics for both routing paths are reported in Table V.

TABLE V
PIPELINE TIMING METRICS

Metric	Classical	Quantum
Avg. Analysis Time	731 ms	1,355 ms
Avg. Decision Time	12 ms	10 ms
Avg. Total Pipeline Time	1,222 ms	1,770 ms
Language Detection Confidence	0.942	0.966

Quantum workloads require approximately 1.85× longer analysis time due to additional quantum metrics extraction (circuit parsing, entanglement scoring). Decision latency remains consistently below 15ms across all workloads, well within the 500ms requirement specified in the system design. Total pipeline throughput averages 1.5 seconds end-to-end.

F. Observations on Current Quantum Limitations

Our benchmark data reflects the limitations of current NISQ hardware:

1) **No workload is currently cheaper on quantum hardware.** Even the smallest quantum circuits (∼3 qubits) cost approximately $2 USD on real hardware versus $0.001 classically. This is due to per-shot costs, queue times, and base job submission fees.
2) **Quantum hardware is consistently slower for small problems.** A single quantum gate takes approximately 100–500 nanoseconds, and 1,024 measurement shots are required for statistical results. Classical CPUs perform billions of operations per second with deterministic results.
3) **The value proposition is forward-looking.** Quantum advantages will emerge when:
   - Error-corrected quantum computers arrive (estimated 2028–2035)
   - Problem sizes exceed 50+ qubits for exact simulation
   - Algorithmic speedups (exponential for factorization, quadratic for search) outweigh hardware cost premiums
4) **The routing framework is ready.** When quantum hardware crosses the advantage threshold, Nexar's pipeline will automatically route eligible workloads to quantum backends – the decision infrastructure is in place.

G. End-to-End Case Studies

Case A: Linear Search → Classical Routing. A Python `for`-loop searching an unsorted 1,024-element array is submitted. The Code Analysis Engine detects $O(N)$ time complexity and CodeBERT classifies it as "search" (confidence 0.92). The AI Code Converter maps it to a 10-qubit Grover circuit. However, the cost analyzer estimates $0.001 classical versus $1.95 quantum. The weighted vote routes to Classical (confidence 0.88): the $O(\sqrt{N})$ speedup does not justify the 1,950× cost premium at this problem size, where $N = 1{,}024$ is the number of elements.

Case B: 120-Qubit VQE → Quantum Override. A 120-qubit VQE ansatz with three variational layers and a nearest-neighbor CNOT ladder is submitted. Analysis extracts `qubits=120`, `circuit_depth=14`, `cx_gate_ratio=0.33`. The rule engine calculates that classical state-vector simulation would require $2^{120} \times 16$ bytes ($\sim 10^{31}$ MB), triggering a `FORCE_QUANTUM` override (confidence 1.0) that bypasses ML voting entirely. The HAL routes the job to a 127-qubit IBM Eagle processor. An example machine-readable decision trace is shown in Fig. 7.

The corresponding user-facing dashboard rendering is shown in Fig. 8.

INPUT

```
Workload: QAOA Balanced Graph Partitioning
Language: Qiskit (confidence: 0.97)
LOC: 236  Algorithm: QAOA   Problem: Optimization
```

EXTRACTED FEATURES

Qubits:	20	Circuit Depth:	108
Gate Count:	130	CX Gate Ratio:	0.338
Superposition:	1.000	Entanglement:	0.508
Time Complexity:	$O(n^2)$	Memory:	8 MB

DECISION **QUANTUM** (conf: 0.805)

Rationale: [RULE OVERRIDE] High superposition potential (1.00); Strong entanglement (0.51)

Est. Cost: $2.24 — Est. Time: 274ms

Classical Alt: $0.001 — Pipeline: 4.8s

Fig. 7. Example system output for a 20-qubit QAOA graph partitioning workload. The pipeline extracts quantum features, detects high superposition and entanglement, and routes to quantum hardware via rule override at 0.805 confidence despite the $2,240× cost premium over classical execution.

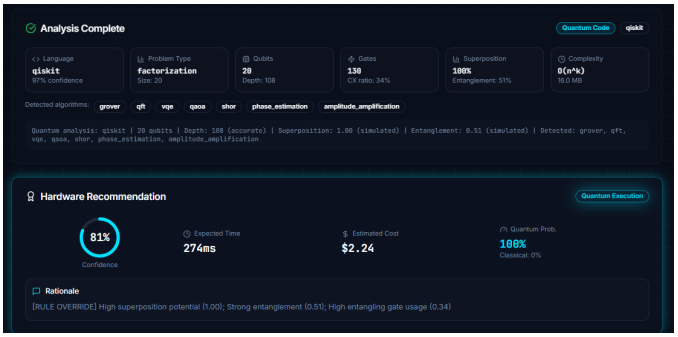

Fig. 8. Nexar web interface showing analysis and routing results for the 20-qubit QAOA graph partitioning workload from Fig. 7.

H. System Latency and Cold Starts

Cloud Run's scale-to-zero model introduces cold start penalties of 10–15 seconds when ML models (CodeT5 safetensors, CodeBERT, scikit-learn ensembles) must be loaded from Cloud Storage into container memory. Once warm, the full pipeline averages 1.5 seconds end-to-end (analysis 731–1,355ms, decision <15ms). The API Gateway enforces a 30-second timeout per upstream call; if a model service fails to respond, the pipeline degrades gracefully to rule-based heuristics.

IX. Discussion

A. Decision Engine Effectiveness

The hybrid decision approach – combining ML predictions with physics-aware rules and cost analysis – proves effective across the workload spectrum. The rule system's override capability is critical: it prevents the ML model from recommending quantum execution for zero-qubit classical problems and enforces hardware limits. The 87% ML accuracy, while not perfect, is compensated by the rule system's deterministic constraints, resulting in appropriate routing for all 100 benchmark workloads. This aligns with the reference architecture for hybrid quantum-classical decision optimization proposed by Chinnaraju [26].

The weighted voting scheme (50% ML, 35% rules, 15% cost) reflects the current state of quantum computing where physics constraints and hardware limitations are often more decisive than learned patterns. As quantum hardware matures and the decision boundary becomes more nuanced, the ML component's weight may increase. Alternative routing approaches such as interference-based mixture-of-experts [27] may also be explored.

B. The Quantum Cost Paradox

Our results reveal what we term the "Quantum Cost Paradox" in the NISQ era: the decision engine correctly identifies workloads that would theoretically benefit from quantum execution, but the actual cost on current hardware is always higher than classical. This paradox resolves at scale – for 120+ qubit problems, quantum is the only viable option regardless of cost. The platform's transparency in presenting both theoretical advantages and practical cost is a feature that aids informed decision-making.

C. Limitations

1) **Synthetic Training Data:** The ML model is trained on synthetic problem instances rather than real quantum execution traces. As real quantum benchmark data becomes available, retraining the models on empirical data should improve routing accuracy.
2) **Static Cost Models:** Provider pricing is encoded as configuration parameters. Real-time pricing integration would improve cost accuracy.
3) **Limited Problem Types:** The current system focuses on optimization, search, simulation, and factorization. Extending to quantum chemistry, quantum machine learning, and cryptographic workloads would broaden applicability.
4) **NISQ-Era Constraints:** The system is designed for current NISQ hardware. Fault-tolerant quantum computing will require updated decision logic, cost models, and hardware constraints.

X. Conclusion

We presented Nexar, an intelligent quantum-classical workload routing platform that addresses the fundamental challenge of execution target selection in the NISQ era. Through the integration of automated code analysis, hybrid ML-and-rules-based decision-making, and provider-agnostic quantum execution, the platform provides a complete pipeline from source code submission to hardware recommendation and job execution.

Our evaluation across 100 diverse workloads demonstrates that the system achieves correct routing decisions across

all workload categories, with pipeline latency averaging 1.5 seconds and decision inference under 15ms. The cost analysis identifies a clear quantum-classical crossover at approximately 50–57 qubits, beyond which classical simulation becomes economically and physically intractable.

The platform's transparent assessment of current NISQ limitations – where no workload is yet cheaper on quantum hardware for small circuits – combined with its readiness to route workloads when quantum advantage materializes, positions Nexar as both a practical decision-support tool for today and an automated routing infrastructure for the fault-tolerant quantum era.

XI. Future Work

1) **Real Quantum Execution Benchmarks:** Execute benchmark workloads on real IBM Quantum and AWS Braket devices to validate cost and timing estimates against actual hardware performance.
2) **Adaptive Model Retraining:** Implement a feedback loop where execution results are used to continuously retrain the decision model, improving routing accuracy over time.
3) **Error Mitigation Integration:** Incorporate quantum error mitigation techniques (zero-noise extrapolation, probabilistic error cancellation) into the cost model to reflect the true cost of obtaining reliable quantum results.
4) **Multi-Objective Optimization:** Extend the decision framework to jointly optimize for cost, time, accuracy, and carbon footprint, supporting user-defined priority weights.
5) **Fault-Tolerant Readiness:** Develop decision logic and cost models for fault-tolerant quantum computers, preparing the platform for the post-NISQ era.
6) **Dynamic Pricing Integration:** The current cost analyzer relies on static provider parameters ($0.30 base fee, $1.60/s QPU rate, $0.00145/shot). As quantum cloud marketplaces mature, these should be replaced with real-time pricing oracles that query provider APIs for current spot-instance availability, queue-length-dependent pricing, and regional cost variations. This would improve cost estimate fidelity and enable the decision engine to exploit transient pricing windows where quantum execution becomes cost-competitive at lower qubit counts.

References

[1] J. Preskill, "Quantum computing in the NISQ era and beyond," *Quantum*, vol. 2, p. 79, 2018.

[2] B. Weder, J. Barzen, F. Leymann, and M. Salm, "Automated quantum hardware selection for quantum workflows," *Electronics*, vol. 10, no. 8, p. 984, Apr. 2021.

[3] IBM Quantum, "Qiskit Runtime," IBM Research, 2023. [Online]. Available: https://quantum.ibm.com/

[4] Amazon Web Services, "Amazon Braket—quantum computing service," 2023. [Online]. Available: https://aws.amazon.com/braket/

[5] Microsoft, "Azure Quantum," 2023. [Online]. Available: https://azure.microsoft.com/en-us/products/quantum

[6] W. Tang *et al.*, "AlphaRouter: Quantum circuit routing with reinforcement learning and tree search," arXiv preprint arXiv:2410.05115, Oct. 2024.

[7] X. Zhan *et al.*, "A full stack framework for high performance quantum-classical computing," arXiv preprint arXiv:2510.20128, Oct. 2025.

[8] M. G. Lammers, F. H. Holik, and A. Fernandez, "Quantum resource management in the NISQ era: Challenges, vision, and a runtime framework," arXiv preprint arXiv:2508.19276, Aug. 2025.

[9] R. Hai, S.-H. Hung, T. Coopmans, and F. Geerts, "Data management in the noisy intermediate-scale quantum era," *Proc. VLDB Endow.*, vol. 18, no. 6, 2025.

[10] M. Beverland *et al.*, "Assessing requirements to scale to practical quantum advantage," arXiv preprint arXiv:2211.07629, 2022.

[11] N. Quetschlich, M. Soeken, P. Murali, and R. Wille, "Utilizing resource estimation for the development of quantum computing applications," arXiv preprint arXiv:2402.12434, Aug. 2024.

[12] M. Kashif, A. Marchisio, and M. Shafique, "Closing the loop: Resource-aware hybrid NAS guided by analytical and hardware-calibrated quantum cost modeling," arXiv preprint arXiv:2603.00625, Mar. 2026.

[13] A. JavadiAbhari *et al.*, "ScaffCC: Scalable compilation and analysis of quantum programs," *Parallel Comput.*, vol. 45, pp. 2–17, 2015.

[14] Z. Feng *et al.*, "CodeBERT: A pre-trained model for programming and natural languages," in *Findings of the Association for Computational Linguistics: EMNLP*, 2020, pp. 1536–1547.

[15] S. Afrose, V. Leyton-Ortega, and T. Humble, "Learning-based quantum compilation: Translating QASM to QIR with CodeBERT," in *Proc. IEEE Int. Conf. Quantum Comput. Eng. (QCE)*, 2025, pp. 576–577.

[16] S. Aaronson and A. Arkhipov, "The computational complexity of linear optics," in *Proc. 43rd Annu. ACM Symp. Theory Comput. (STOC)*, 2011, pp. 333–342.

[17] F. Arute *et al.*, "Quantum supremacy using a programmable superconducting processor," *Nature*, vol. 574, pp. 505–510, 2019.

[18] T. J. McCabe, "A complexity measure," *IEEE Trans. Softw. Eng.*, vol. SE-2, no. 4, pp. 308–320, Dec. 1976.

[19] L. Vinkhuijzen, T. Coopmans, D. Elkouss, V. Dunjko, and A. Laarman, "LIMDD: A decision diagram for simulation of quantum computing including stabilizer states," *Quantum*, vol. 7, p. 1108, Sep. 2023.

[20] S. Simsek, A. Ceran, and F. Yildiz, "Quantum circuit optimization using machine learning," arXiv preprint arXiv:2305.06585, May 2023.

[21] E. Farhi, J. Goldstone, and S. Gutmann, "A quantum approximate optimization algorithm," arXiv preprint arXiv:1411.4028, 2014.

[22] Y. Wang *et al.*, "CodeT5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation," arXiv preprint arXiv:2109.00859, Sep. 2021.

[23] L. K. Grover, "A fast quantum mechanical algorithm for database search," in *Proc. 28th Annu. ACM Symp. Theory Comput. (STOC)*, 1996, pp. 212–219.

[24] P. W. Shor, "Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer," *SIAM J. Comput.*, vol. 26, no. 5, pp. 1484–1509, 1997.

[25] A. Kandala *et al.*, "Hardware-efficient variational quantum eigensolver for small molecules and quantum magnets," *Nature*, vol. 549, pp. 242–246, 2017.

[26] A. Chinnaraju, "Hybrid quantum-classical intelligence for business: A reference architecture for predictive analytics and decision optimization," *J. Adv. Future Res.*, vol. 3, no. 12, Dec. 2025.

[27] R. Heddad and L. Bouanane, "Hybrid quantum-classical mixture of experts: Unlocking topological advantage via interference-based routing," arXiv preprint arXiv:2512.22296, Dec. 2025.